
The connectivity problem

ALGORITHMS AND DATA STRUCTURES

PRACTICAL 1

Group 39

Angelina Podolako

Maria Thiele

s1125886

s1130739

1 Explanation of the Algorithm

Given a set of edges between vertices, the goal of this algorithm is to determine the minimum number of edges needed to connect all vertices with one another. There are three edge types and it has to be possible to move through the entire network of vertices by using either only "pedestrian" edges or only "bus" edges. The third edge type "both" defines edges that count to both "pedestrian" and "bus" edges.

1.1 Initialization

1.1.1 Reading in the data

The program reads in a file over the standard input. This file contains the data and its format is known. First, the data is preprocessed by replacing every line break with a space. As a result, the entire content of the file ends up in the first line. The first two integer values always give the total amount of vertices and the total amount of edges. After these first integers the next 3 consecutive integers form a set (u, v, type) with (u, v) defining an edge between the vertices u and v and type being the identifier for the edge type. As explained above there are three edge types (pedestrian, bus, both). After each vertex is initialized as a Vertex object (see subsection 1.1.2) every edge is sorted into a list corresponding to the type of the edge. This results in a list that has other lists as elements. Those smaller lists contain two Vertex objects which represent the vertices between which the edge exists. The type of the edge does not need to be explicitly stored because it is already indirectly stored based on the list into which an edge was sorted.

1.1.2 Initializing the vertices through the Vertex class

Since the total number of vertices is known after reading in the data file, the next step is the initialization of a separate Vertex object for each of those vertices. The class Vertex is used to create such an object. Each vertex has a number and a component. Because the vertices are characterized through numbers in the data file, the number of a vertex is the number in the data file. In other words, there is a vertex with a corresponding number for every integer between 0 and the amount of total vertices. The component attribute of each vertex is initialized with the integer -1. This shows that the vertex is not yet part of a component. Since at the beginning of the algorithm no edge has been added to the graph, every vertex does not belong to a component.

1.1.3 Creating the component structure

After having initialized the Vertex objects and the lists of edges, the overall component structure is created. The data structure used for this is a dictionary. The keys of this dictionary are the identifiers of the different components and the values are the list of vertices that belong to a component. At

first, there is only one key, namely the integer -1 and its value is a list of all vertices. This is due to the fact that at the start no vertex belongs to a specific component.

1.2 Algorithm Logic

The main idea of the algorithm is to group the vertices into components. Each component consists of vertices that are connected among each other. If there is an edge between two vertices of different components, then the algorithm moves the vertices into the same component.

Three cases can be differentiated:

1. both vertices do not belong to a component yet
⇒ the algorithm creates a new component including only these two vertices
2. one vertex does not yet belong to a component
⇒ the vertex that does not belong to a component yet is added to the component of the other vertex
3. both vertices belong to (different) components
⇒ the vertex belonging to the smaller component is determined and every vertex in this component is added to the component of the other vertex

This main idea of the algorithm is also provided in Algorithm 1. Note that this is just an outline and the actual code is more detailed and also considers some optimization strategies for efficiency purposes.

Before any components are created, the algorithm checks whether it is even possible to construct a graph that connects all vertices. In case there are fewer edges than there are vertices, this is impossible, hence the algorithm returns "-1". Because edges of type "both" are preferable over edges of type "pedestrian" or type "bus", the algorithm first adds edges of type "both" to the graph. An edge is added when the components change, so whenever two vertices do not already belong to the same component. Should the graph already be fully connected after adding edges of type "both", the algorithm returns the minimum number of edges used.

In most cases edges of type "both" will not be sufficient to connect all vertices. In this case, the algorithm determines which edge type (between pedestrian and bus) has fewer edges. Based on the components already constructed through "both" edges, the algorithm continues to try and connect these components with edges from the edge type that had fewer edges. Should there be more than 1 component after going through all the edges of this type, then some components can not be connected by this edge type. As a result, the problem cannot be solved and the algorithm returns "-1".

The pseudocode version of how components are connected can be found in Algorithm 2 but again, this is just a rough outline of the actual code.

If all components could be connected, the algorithm resets the state of the components to the state after having added the edges of type "both" and proceeds to try and connect these components with the remaining edge type. Again, if there are more than 1 components, the program returns with "-1" because there is no solution to the problem. In case there is only one component left, the algorithm has found a solution and returns the sum of all edges that were used to create this graph.

1.3 Returning the Answer

The algorithm either terminates with the string "-1" or with a string of a positive integer value. "-1" is returned when no solution could be found. This can happen at various moments in the algorithm. First, the total amount of vertices and the total amount of edges is compared. In case there are fewer edges than vertices ($E < V$) then it is not possible to create a graph in which all vertices are connected, hence no solution to the problem can be found. Second, "-1" is also returned when no connected graph can be created using only edges for pedestrians (types "pedestrian" and "both") or when no connected graph can be created using edges for buses (types "bus")

Algorithm 1 Main algorithm

```
1: // Step 1 : read in the data and create data structures
2: data = read(data file)
3: total_vertices = get_vertices(data)
4: total_edges = get_edges(data)
5:
6: if total_edges < total_vertices - 1 then
7:     return "-1"
8:
9: for every vertex in total_vertices do
10:     create vertex object by setting:
11:     vertex.component = -1
12:     vertex.number = number of vertex
13:
14: for every edge in data do
15:     if edge type == "pedestrian" then
16:         add edge to connectionList_pedestrian
17:     else
18:         if edge type == "bus" then
19:             add edge to connectionList_bus
20:         else
21:             add edge to connectionList_both
22:
23:
24: // Step 2 : run the core algorithm and save the minimum number of edges
25: result = create_MST(vertices, connectionList_pedestrian, connectionList_bus, connectionList_both)
26:
27:
28: // Step 3 : Determine the output
29: if result == -1 then
30:     print("-1")
31: else
32:     print(total_edges - result)
```

and "both"). The first edges added to the graph are edges of type "both". For each component the vertices of that component are all reachable by foot and by bus. Therefore, the algorithm uses this state of components to then check if the individual components can now be connected using only "pedestrian" edges and using only "bus" edges. If both of these succeed, there is a solution to the problem. If only one of them does not succeed, the algorithm returns "-1".

In case there is a solution to the problem, the minimal number of edges needed for a solution is computed and based on this and the total number of edges from the data, the algorithm returns the amount of edges that were removed. The earliest return of such an amount is after creating the graph with only "both" edges.

In general, after applying each type of connections the algorithm checks if a decision about the solution can already be made and should this be the case, the algorithm terminates and returns the corresponding result.

Algorithm 2 Helper functions

```
function CREATE_MST(vertices, pedestrianCon, busCon, bothCon)
    // create a dictionary that contains all components, at first no vertex has a component
    componentsDictionary = dictionary with components

    // run the algorithm per connection type and increase the counter
    edgeCounter0 = run_for_connection(componentsDictionary, bothCon)

    // save the state of the componentsDictionary
    pedDictionary = componentsDictionary
    busDictionary = componentsDictionary

    // Determine whether busCon or pedestrianCon have fewer amount of edges
    if len(busCon) < len(pedestrianCon) then
        smallerCon = busCon
        largerCon = pedestrianCon
        smallerDictionary = busDictionary
        largerDictionary = pedDictionary
    else
        smallerCon = pedestrianCon
        largerCon = busCon
        smallerDictionary = pedDictionary
        largerDictionary = busDictionary

    // run the algorithm for the remaining connections
    edgeCounter1 = run_for_connection(smallerDictionary, smallerCon)
    edgeCounter2 = run_for_connection(largerDictionary, largerCon)

    // Determine output
    if edgeCounter0 and edgeCounter1 and edgeCounter2 != -1 then
        return edgeCounter0 + edgeCounter1 + edgeCounter2
    else
        return -1
function RUN_FOR_CONNECTION(dictionary, connections)
    // count edges
    usedEdges = 0

    // Loop through all edges in the connections list and put u and v in the same component
    for edge(u, v) in connections do
        if u and v not in same component then
            move_into_sameComponent(u, v)
            usedEdges += 1
    return usedEdges
```

1.4 Optimizations

The first optimization is checking if the total amount of vertices and the total amount of edges allow for a solution.

If that is not the case, namely when $|E| < |V| - 1$, the algorithm terminates right after reading in the input.

A second optimization is first dealing with only the edges of type "both" and using this as a basis for when the other two types are run. After each run of a new edge type the algorithm checks if either a solution is already found (namely when using only "both" edges already connects all vertices) or if a

solution cannot exist (namely when either "pedestrian" or "bus" connections alone cannot connect the components formed by "both" connections).

Thirdly, the order in which the "pedestrian" and the "bus" connections are run is based on the amount of edges per type. If there are less "pedestrian" edges, the algorithm first checks if the components formed by "both" connections can be connected by "pedestrian" edges and since there are fewer "pedestrian" edges than "bus" edges there is a higher probability that there are not enough edges to connect all components (and in that case the algorithm terminates without having to run through the "bus" connections).

2 Correctness Analysis

2.1 Correct termination with "-1"

The algorithm has to terminate whenever no solution can be found to solve the problem. This is either the case because there are too few edges to solve the problem or because not every vertex is connected to the others by only foot or bus.

1. Termination for insufficient amount of edges

The algorithm checks if there are at least as many edges as vertices and if not, it terminates with "-1"

2. Insufficient connection

First, the algorithm goes through all edges of type "both". The resulting components consist of vertices that are connected through foot and bus routes (definition of type "both").

Per remaining type the algorithm uses the components found for "both" connections and tries to connect these components by only "pedestrian" edges and by only using "bus" edges. If one of these fails, there is no solution because some components will not be reachable by one specific type and the algorithm returns "-1".

2.2 Correct termination with "removed number of edges"

When there is a solution the algorithm has to return the number of edges that are not necessary to solve the problem.

This works as the edges are added one by one and only if they connect two vertices that are not yet in the same component. For vertices in different components, the algorithm adds one of the vertices v and all other vertices in that vertex component (if applicable) to the other vertices component.

This leads to a minimally connected structure, meaning all vertices in a component are connected but the amount of edges is minimal. If there is an edge that would connect two vertices that are in the same component, the edge is ignored. By subtracting the needed amount of edges from the total amount, the removed edges are computed.

2.3 Correct manipulation of components

At the start, no vertices are in a component and the attribute `vertex.component` is set to -1. In the component dictionary there is a key -1 that contains a list of all vertices not belonging to a component, so at the start this is a list of all vertices.

1. At least one vertex is already in a component

When a vertex is moved to a new component, the key in the dictionary to whose values the vertex is added is used to update the `vertex.component` attribute. So if a vertex v was in component 3 and is then moved to component 4, `v.component` gets updated to 4. If said vertex was in a component before, the `v.component` attribute for all vertices of that component get updated to the new component. Every vertex that changed the component gets removed from the old component.

2. Both vertices have no component If both vertices have `vertex.component = -1` (so no component) then the first of the two is used to create a new key in the dictionary based on the vertex number and both vertices are added to that key and removed from the values for key=-1.

So the component of each vertex is updated when the vertex gets added to a new component.

3 Complexity Analysis

Complexity analysis is performed to evaluate the efficiency of the implemented algorithm in terms of time and space complexity. The terms used are the number of vertices V (or number of scenes) and edges E (number of roads).

- Reading and parsing input data: $O(V + E)$
When reading in the input, newline characters are replaced with spaces and the text is split into individual elements. Each value is processed once, so the data reading and parsing time has linear complexity $O(V + E)$.
- Creating vertex objects: $O(V)$
This happens in a loop that iterates over all vertices, requiring $O(V)$ in time and memory.
- Separation of edges by type: $O(E)$
Each edge is processed once and sorted into a respective list, so this operation is linear in the number of edges: $O(E)$.
- Constructing the MST: $O(V \times E)$
Here, the dictionary `component_dict` is used to store connected components, where the keys are the identifiers of the components and the values are lists of vertices belonging to these components. When adding edges, the algorithm checks whether the vertices belong to the same component. If not, the components are merged, which requires transferring all vertices from one component to another. This is a simplified version of a union-find algorithm (like Disjoint Set Union), where connecting vertices is done manually through lists. In the worst case, a merge operation can involve all vertices, so a single union operation takes $O(V)$ time. Since such operations are performed for each edge (in the worst case), the time complexity of the main part of the algorithm is estimated as $O(V \times E)$.
- Checking components: $O(V)$
After completing the graph construction, the algorithm performs several linear checks of the component states and calculates the number of edges removed. They take $O(V)$ time.
- Total time complexity: $O(V \times E)$

This means that the execution time grows proportionally to the product of the number of vertices and the number of edges.

4 Reflection

Each team member came up with a slightly different algorithm, one based on vertices and one based on a Disjoint Set Union (DSU) structure. While the (DSU) structure seems to run more efficiently for larger input data, the vertex based structure seems to be faster for smaller inputs. Trying to combine both versions in a way that the vertex structure is used for smaller inputs and the DSU structure is used for larger inputs resulted in significantly longer overall runtimes for nearly every test case. This seems counterintuitive and we sadly did not manage to resolve this. The decision to use the vertex based algorithm was made because it was slightly faster than the DSU implementation (on DomJudge 4,439s for the vertex based one and 4,482s for the DSU based one).